Operadores Lógicos 🧠

En esta lección vamos a aprender sobre un nuevo tipo de operadores, los `operadores lógicos`. Estos son operadores que nos permiten hacer preguntas a la computadora, para las cuales siempre vamos a obtener una respuesta binaria: `True` o `False`.

Este concepto va a ser la base para llevar nuestros programas al siguiente nivel, ya que vamos a poder tomar decisiones en base a ciertas condiciones.

Igual a `==`

Este primer operador revisa si dos valores son **idénticos**, tomando en cuenta su valor y su tipo de dato. Es importante no confundirlo con el operador de asignación (`=`).

Copiar

```
print(10 == 10)

user_password = "Cheese123"

print(user_password == "Cheese123")
print(user_password == "NotCheese123")
```

✓
True
True
False

Distinto a `!=`

Este operador revisa si dos valores son **diferentes**. Es el opuesto del operador Igual a `==`. Retorna `True` si NO son iguales.

```python
print("Hola" != "Mundo")

value_1 = 25
value_2 = 25

print(value_1 != value_2)
```

Copiar

True
False

Mayor a `>`

Revisa si un número es **mayor** a otro. Al igual que como lo aprendimos en matemática básica, la apertura de la flecha apunta hacia el valor mayor para que el resultado sea `True`.

💡 **Nota importante**: cuando ambos valores son iguales retorna `False`. Esto es porque el operador `>` requiere que el valor de la izquierda sea estrictamente mayor para que sea `True`.

```
grade_1 = 95
grade_2 = 70

print(grade_1 > 70)
print(grade_2 > 70)
print(grade_2 > grade_1)
```

Copiar

```
True
False
False
```

Mayor o igual a `>=`

Revisa si un numero es **mayor** o **igual** a otro. Muy similar al anterior, pero en este caso si pueden ser iguales y por ende retorna `True`. En que este caso, cuando ambos valores son iguales retorna `True`.

```
grade_1 = 95
grade_2 = 70

print(grade_1 >= 70)
print(grade_2 >= 70)
print(grade_2 >= grade_1)
```

Copiar

```
True
True
False
```

Menor a `<`

Revisa si un número es **menor** a otro. La punta de la flecha apunta al valor que esperamos que sea el más pequeño.

💡 **Nota importante**: cuando ambos valores son iguales retorna `False`

Copiar

```
account_balance = 50
item_price = 100

print(account_balance < 100)
print(item_price < 100)
print(item_price < account_balance)
```

✓
True
False
False

Menor o igual a `<=`

Revisa si un número es **menor** o **igual** a otro. Muy similar al anterior, pero en este caso si pueden ser iguales y por ende retorna `True`.

```python
account_balance = 50
item_price = 100

print(account_balance <= 100)
print(item_price <= 100)
print(item_price <= account_balance)
```

Copiar

```
True
True
False
```

Negación `not`

Invierte un valor a su **opuesto**, este es usado principalmente en `booleans`. Si el valor es `True` pasa a `False` y viceversa.

```python
# Invertir un booleano
print(not True)
print(not False)

# Invertir el valor de una variable
is_adult = True
print(not is_adult)

# Invertir el resultado de una operación lógica
print(not (5 > 10))
```

Copiar

- ✓ False
- True
- False
- True

Resumen ✍️

Operación	Símbolo	Descripción	Ejemplos
Igual a	`==`	Revisa si dos valores son idénticos.	`10 == 10 → True` `10 == "10" → False`
Distinto a	`!=`	Revisa si dos valores son diferentes.	`10 != 10 → False` `"Key" != "Lock" → True`
Mayor a	`>`	Revisa si un número es mayor a otro.	`90 > 40 → True` `50 > 50 → False`
Mayor o igual a	`>=`	Revisa si un número es mayor o igual a otro.	`90 >= 40 → True` `50 >= 50 → True`
Menor a	`<`	Revisa si un número es menor a otro.	`90 < 40 → False` `50 < 50 → False`
Menor o igual a	`<=`	Revisa si un número es menor o igual a otro.	`90 <= 40 → False` `50 <= 50 → True`

Operación	Símbolo	Descripción	Ejemplos

Negación	`not`	Invierte el valor de un booleano.	`not True → False`
			`not False → True`
			`not (5 > 10) → True`

Combinando Condiciones: `and` y `or`

A medida que avancemos en nuestro camino, nos encontraremos con situaciones donde una sola pregunta no es suficiente. Para abordar escenarios más complejos, necesitamos la capacidad de unir varias condiciones lógicas. Aquí es donde entran en juego los operadores `and` y `or`.

Operador `and`

Este operador analiza si **todas** las condiciones unidas son `True`. Si al menos una de ellas es `False`, el resultado final será `False`. Es muy estricto, todo debe cumplirse.

```python
is_weekend = True
is_sunny = True

# Ambas condiciones se cumplen
print(is_weekend and is_sunny)

is_weekend = True
is_sunny = False

# Una condición falla
print(is_weekend and is_sunny)
```

Copiar

True
False

Operador `or`

Este operador es más flexible, analiza si **al menos una** de las condiciones es `True`. El único caso en el que devuelve `False` es cuando **todas** las condiciones son `False`.

```python
has_coffee = True
has_tea = False

# Al menos una es verdadera
print(has_coffee or has_tea)

has_coffee = False
has_tea = False

# Ninguna es verdadera
print(has_coffee or has_tea)
```

Copiar

✓ True
False

Prioridad con Paréntesis `()`

En matemáticas, sabemos que la multiplicación se resuelve antes que la suma. De manera similar, en programación los operadores lógicos tienen un orden de prioridad, se evalúa primero el `and` y luego el `or`.

Pero nosotros podemos usar paréntesis `()` para agrupar condiciones y asegurar que la lógica sea la correcta.

🎉 Pensemos en un sistema de entrada a una fiesta. Para entrar, una persona necesita:

Tener su entrada `Y` además, cumplir con una de dos condiciones:

ser mayor de edad `O` tener permiso de sus padres.

Si no usamos paréntesis, el resultado puede ser el incorrecto. Veamos el caso sin paréntesis:

```python
has_ticket = False
is_adult = True
has_permission = True

# (False and True) or True  ->  False or True  ->  True
print(has_ticket and is_adult or has_permission)
```

Copiar

✓ True

Por el orden de los operadores se ejecuta primero el `and`, por ende ¡tenemos un error! En cambio, si implementamos los paréntesis, resolvemos el problema:

```python
has_ticket = False
is_adult = True
has_permission = True

# Forzamos a evaluar primero el grupo (edad o permiso)
# False and (True or True)  ->  False and True  ->  False
print(has_ticket and (is_adult or has_permission))
```

Copiar

✓ False